NHS Technical Report

By Luke Whitmore

Word count 1,100 (excluding titles and appendices)

Business Problem/ Context

The NHS is considering expanding its primary care service capacity to meet the needs of its patients. However, it must first understand how well its existing capacity is being utilised before deciding whether to increase this or focus on making better use of existing resources and infrastructure.

The NHS's two high-level business questions are:

1) Has there been adequate staff and capacity?
2) What was the actual utilisation of resources?

Internal data has been provided to support this analysis. The focus will be on metrics relating to appointment capacity, demand, and capacity utilisation, on demand by appointment mode, and on appointments with the status "did not attend", commonly referred to as "DNAs".

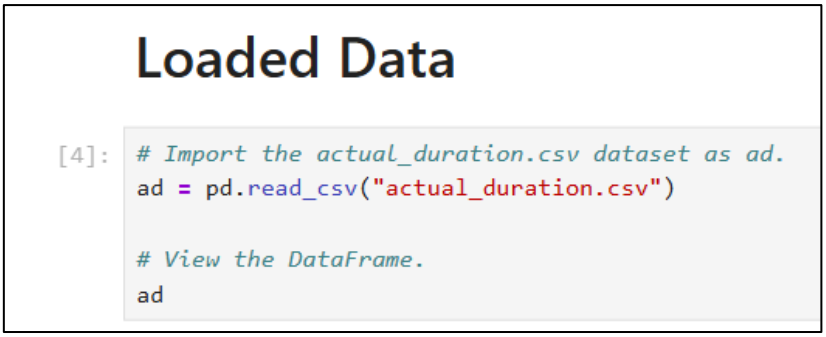
Setup of Jupyter Notebook

The pandas, NumPy, Seaborn, Matplotlib, and Matplotlib.pyplot libraries were imported for use in this analysis.

The below user-defined functions were created and implemented to reduce the length and repetition of frequently used code (see the Jupyter Notebook for the code).

User-defined Function Name	User-defined Function Purpose
inspect_and_validate_data(df)	Determine the metadata and perform descriptive statistics
perform_outlier_analysis_and_generate_boxplot	Create a boxplot
clean_label	Change axis labels to proper case
generate_time_series_plot	Create a timeseries plot
generate_standard_bar_plot_days	Create a bar plot using counts of days
generate_standard_bar_plot_appts	Create a bar plot using counts of appointments
generate_stacked_bar_plot	Create a stacked bar plot

The actual_duration, appointments_regional, national_categories, and tweets CSV files were loaded for analysis and given aliases to make the code more pythonic.



Assumptions and Limitations

This analysis relates to the period August 2021 to June 2022, because the nc dataset only covers this period. The ar dataset was, therefore, filtered to only show data within this range. The ad dataset was not used to because it was deemed to have limited relevance to the business questions.

The NHS informed us that there were 1.2 million appointments available each day nationally (capacity). This figure was multiplied by 30 days to estimate monthly capacity of 36 million appointments. Accuracy is limited because some months do not have 30 days.

The NHS did not inform us of each Integrated Care Board's (ICB) capacity. Therefore, it was assumed that each ICB was responsible for an equal share of national capacity, equalling 1/42, or 28,571 appointments per ICB per day. Accuracy is limited because, in reality, some ICB locations will have had more capacity than others.

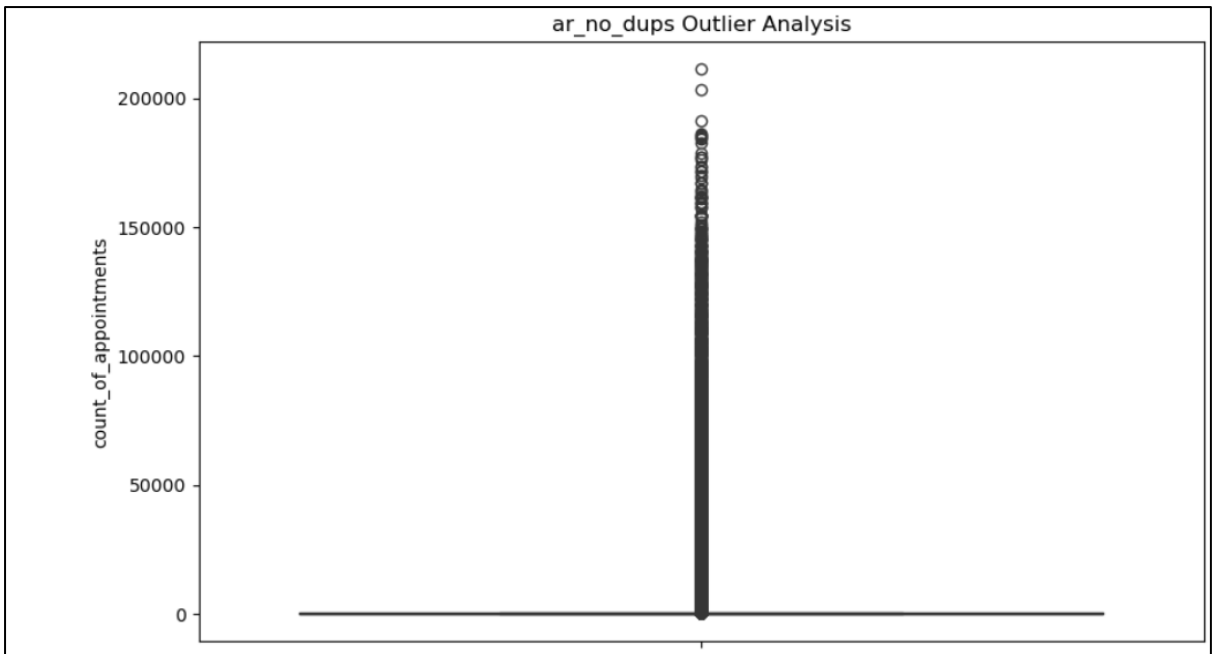
Data Inspection/ Validation, Wrangling and Analysis

The inspect_and_validate_data(df) function was applied to the datasets. There were no missing values in any of the datasets and no duplicate values in the ad or nc datasets. There were 21,604 duplicated rows within the ad dataset, which were removed in a new dataset called ar_no_dups.

```
[32]: # Remove 21604 fully duplicated rows from the ar dataset.
      ar_no_dups = ar.drop_duplicates()

      # Re-inspect and validate the ar dataset.
      inspect_and_validate_data(ar_no_dups)
```

```
Number of unique values per column:
icb_ons_code                          42
appointment_month                     30
appointment_status                     3
hcp_type                               3
appointment_mode                       5
time_between_book_and_appointment      8
count_of_appointments                  22807
dtype: int64

Number of missing values per column:
icb_ons_code                          0
appointment_month                     0
appointment_status                    0
hcp_type                              0
appointment_mode                      0
time_between_book_and_appointment     0
count_of_appointments                 0
dtype: int64

Number of fully duplicated rows:
0
```

Outlier analysis was performed on the datasets. However, all outliers were retained because they were deemed likely to represent legitimate variation in the counts of appointments booked.




```
[33]: {'Q1': np.float64(8.0),
       'Q3': np.float64(332.0),
       'IQR': np.float64(324.0),
       'lower_bound': np.float64(-478.0),
       'upper_bound': np.float64(818.0),
       'outlier_count': 93641}
```

To answer business question 1, the following wrangling and analysis was undertaken:

The nc dataset was grouped by appointment month and then summed by count of appointments. This was required to assess monthly national demand vs. capacity.

Question 2: How many appointments were booked each month nationally (demand)?

```
[46]: # Use the nc dataset to determine the number of appointments booked per month.
      # Use the groupby() function.
      nc.groupby('appointment_month')['count_of_appointments'].sum().reset_index()
```

[46]:

	appointment_month	count_of_appointments
0	2021-08	23852171
1	2021-09	28522501
2	2021-10	30303834
3	2021-11	30405070
4	2021-12	25140776
5	2022-01	25635474
6	2022-02	25355260
7	2022-03	29595038
8	2022-04	23913060
9	2022-05	27495508
10	2022-06	25828078

The average demand per day was then calculated, to determine the proportion on which demand was manageable vs. unmanageable. This was necessary for identifying daily capacity issues.

Question 3: How often was national daily demand manageable within national capacity?

[48]:
```
# Calculate the number of days on which appointments were booked.
nc_num_appt_days = nc['appointment_date'].nunique()

# View the result.
print(f"Number of days on which appointments were booked: {nc_num_appt_days}")

# Group by appointment date and sum all appointments nationally.
daily_nat_dem = nc.groupby('appointment_date')['count_of_appointments'].sum()

# Filter the appointment dates to determine the number of days where national demand was manageable within national capacity.
days_sufficient_nat_cap = daily_nat_dem[daily_nat_dem < daily_nat_cap]

# View the result.
print(f"Number of days with sufficient national capacity to meet demand: {len(days_sufficient_nat_cap)}")

# Calculate the percentage of total days where national demand was manageable within national capacity.
perc_days_sufficient_nat_cap = round((len(days_sufficient_nat_cap) / nc_num_appt_days) * 100, 2)

# View the result.
print(f"Percentage of days with sufficient national capacity to meet demand: {perc_days_sufficient_nat_cap}")
```

```
Number of days on which appointments were booked: 334
Number of days with sufficient national capacity to meet demand: 159
Percentage of days with sufficient national capacity to meet demand: 47.6
```

The names of the ICBs with insufficient capacity were then identified. This was necessary so the NHS could be told where systemic failings were occurring.

Question 8: Which ICB locations did not have enough daily capacity to manage their average daily demand (the average of their total demand over time)?

[53]:
```
# We are not told the proportion of the daily national capacity that each ICB location has locally.
# Therefore, it is assumed that each ICB location has 1/42 of the daily national capacity (an equal share).
# Calculate the average number of appointments booked per day per ICB location.
avg_daily_icb_dem = nc.groupby("icb_location_name")['count_of_appointments'].sum() / nc_num_appt_days

# Filter the ICB locations to determine which did not have enough daily capacity to manage their average daily demand.
icb_loc_insufficient_cap = avg_daily_icb_dem[avg_daily_icb_dem > daily_icb_cap].index.tolist()

# View the result.
print(f"ICB locations with insufficient daily capacity:")
for location in icb_loc_insufficient_cap:
    print(location)
```

```
ICB locations with insufficient daily capacity:
NHS Cheshire and Merseyside ICB
NHS Greater Manchester ICB
NHS North East London ICB
NHS North East and North Cumbria ICB
NHS North West London ICB
NHS West Yorkshire ICB
```

The number of unutilised appointments per month were then calculated for the neighbouring ICBs. This was necessary so the NHS could see where demand could have been shifted to.

Question 10: Which ICB locations had unutilised capacity available and in which months to help manage the demand of ICBs with insufficient capacity?

[55]:
```
# It is assumed that an ICB would only be able to shift demand to another ICB if they belong to the same NHS Region.
# This is because ICBs within the same Region would be more likely to have established contracting and commissioning relationships.
# It would also be easier to shift demand across a smaller geographic distance if patients were expected to travel, e.g., just over a county line.
# The ICBs locations that need to shift demand are based in the London, North East and Yorkshire, and North West Regions.

# List the ICB locations in the London, North East and Yorkshire, and North West Regions with available capacity.
available_cap_icbs = [
    'NHS North Central London ICB',
    'NHS South East London ICB',
    'NHS South West London ICB',
    'NHS Humber and North Yorkshire ICB',
    'NHS South Yorkshire ICB',
    'NHS Lancashire and South Cumbria ICB'
]

# Filter the nc dataset to include only the ICB locations in the London, North East and Yorkshire, and North West Regions with available capacity.
available_cap_icb_subset = nc[nc['icb_location_name'].isin(available_cap_icbs)]

# Group by appointment month and icb location, then sum the count of appointments.
available_cap_icbs_monthly_appt_counts = available_cap_icb_subset.groupby(['appointment_month', 'icb_location_name'])['count_of_appointments'] \
    .sum().reset_index()

# Filter for months where each ICB had unutilised capacity (count of appointments < 857130).
available_cap_icbs_unutilised_capacity = available_cap_icbs_monthly_appt_counts[available_cap_icbs_monthly_appt_counts['count_of_appointments'] \
    < 857130]

# Pivot the data so each ICB location has its own column and all the data can be viewed at the same time.
pivot_available_cap_icbs_unutilised_capacity = available_cap_icbs_unutilised_capacity \
    .pivot(index='appointment_month', columns='icb_location_name', values='count_of_appointments')

# View the results.
pivot_available_cap_icbs_unutilised_capacity
```

[55]:

icb_location_name	NHS Humber and North Yorkshire ICB	NHS Lancashire and South Cumbria ICB	NHS North Central London ICB	NHS South East London ICB	NHS South West London ICB	NHS South Yorkshire ICB
appointment_month						
2021-08	741301.0	701143.0	551815.0	643439.0	586988.0	605346.0
2021-09	NaN	NaN	648829.0	753909.0	687200.0	713143.0
2021-10	NaN	NaN	674149.0	805256.0	713981.0	775761.0
2021-11	NaN	NaN	690124.0	807761.0	723453.0	777943.0
2021-12	784363.0	740330.0	566643.0	656619.0	592494.0	650409.0
2022-01	792499.0	758205.0	586319.0	681122.0	620753.0	657482.0
2022-02	784545.0	747012.0	587932.0	673031.0	622835.0	651106.0
2022-03	NaN	NaN	679626.0	779408.0	718995.0	755471.0
2022-04	742852.0	701515.0	540612.0	634790.0	578361.0	614425.0
2022-05	841625.0	812080.0	632209.0	731005.0	674675.0	699867.0
2022-06	793072.0	752617.0	589700.0	683830.0	635295.0	648462.0

The demand per appointment mode was then calculated for each ICB that had insufficient capacity. This required that the names of the ICBs were mapped from the nc dataset to the ar_no_dups dataset. This was necessary to determine the volume of remote/virtual appointment demand that could have been shifted.

Question 11: Of the ICB locations that did not have enough capacity, how was demand split between appointment modes?

[56]:
```
# Show the number of appointments by appointment mode for each ICB location with insufficient capacity.
# Use the ar_no_dups dataset because this includes the appointment_mode column.
# The ar_no_dups DataFrame doesn't have the icb_location_name columnn, so this must be derived from the nc DataFrame.
# Create a mapping dictionary from the nc DataFrame for the icb_ons_code and icb_location_name columns.
# Use drop_duplicates to get unique pairs of icb_ons_code and icb_location_name.
icb_ons_code_and_location_name_mapping = nc[['icb_ons_code', 'icb_location_name']] \
    .drop_duplicates().set_index('icb_ons_code')['icb_location_name'].to_dict()

# Apply the mapping to the ar_no_dups DataFrame.
ar_no_dups['icb_location_name'] = ar_no_dups['icb_ons_code'].map(icb_ons_code_and_location_name_mapping)

# Filter the ar_no_dups dataset to only include the ICBs with insufficient capacity.
ar_insufficient_cap_icb_subset = ar_no_dups[ar_no_dups['icb_location_name'].isin(insufficient_cap_icbs)]

# Group by appointment mode and icb location, then sum the count of appointments.
insufficient_cap_icbs_appt_counts_by_mode = ar_insufficient_cap_icb_subset \
    .groupby(['appointment_mode', 'icb_location_name'])['count_of_appointments'].sum().reset_index()

# Pivot the data so each ICB location has its own column and all the data can be viewed at the same time.
pivot_insufficient_cap_icbs_appt_counts_by_mode = insufficient_cap_icbs_appt_counts_by_mode \
    .pivot(index=['appointment_mode'], columns='icb_location_name', values='count_of_appointments')

# View the results.
pivot_insufficient_cap_icbs_appt_counts_by_mode
```

[56]:

icb_location_name	NHS Cheshire and Merseyside ICB	NHS Greater Manchester ICB	NHS North East London ICB	NHS North East and North Cumbria ICB	NHS North West London ICB	NHS West Yorkshire ICB
appointment_mode						
Face-to-Face	18548817	18328791	13083807	27540847	16720440	22981972
Home Visit	466241	448518	131708	304467	72832	122044
Telephone	13373030	12362106	10208320	13944722	11363315	11608870
Unknown	578948	2803309	90924	1140979	618918	1119345
Video/Online	97169	61229	79500	123106	605262	242789

Question 12: Of the ICB locations that did not have enough capacity, how much demand did they experience for telephone and video/online appointments?

[57]:
```
# Filter the ar_no_dups dataset to only show appointments for which the appointment modes was either telephone or video/online.
insufficient_cap_icbs_filtered_by_mode = insufficient_cap_icbs_appt_counts_by_mode[insufficient_cap_icbs_appt_counts_by_mode['appointment_mode'] \
    .isin(['Telephone', 'Video/Online'])]

# Pivot the filtered data.
pivot_insufficient_cap_icbs_filtered_by_mode = insufficient_cap_icbs_filtered_by_mode \
    .pivot(index=['appointment_mode'], columns='icb_location_name', values='count_of_appointments')

# View the results.
pivot_insufficient_cap_icbs_filtered_by_mode
```

[57]:

icb_location_name	NHS Cheshire and Merseyside ICB	NHS Greater Manchester ICB	NHS North East London ICB	NHS North East and North Cumbria ICB	NHS North West London ICB	NHS West Yorkshire ICB
appointment_mode						
Telephone	13373030	12362106	10208320	13944722	11363315	11608870
Video/Online	97169	61229	79500	123106	605262	242789

To answer business question 2, the following wrangling and analysis was undertaken:

The ar_no_dups dataset was filtered by date, and then daily national demand was divided by capacity to determine utilisation.

Question 2: What was the average daily national appointment capacity utilisation (%) for the period August 2021 to June 2022?

[60]:
```
# Calculate the average number of appointments booked per day nationally between August 2021 to June 2022.
# Use the nc_num_appt_days DataFrame for the total number of appointment dates.
avg_daily_nat_dem = ar_no_dups_aug21_to_jun22['count_of_appointments'].sum() / nc_num_appt_days

# Create a new DataFrame to hold the average daily national capacity utilisation calculation.
avg_daily_nat_cap_utilisation_aug21_to_jun22 = avg_daily_nat_dem / daily_nat_cap

# View the result.
print(f"Average daily national appointment capacity utilisation (August 2021 - June 2022): {avg_daily_nat_cap_utilisation_aug21_to_jun22 * 100:.2f}%")
```

```
Average daily national appointment capacity utilisation (August 2021 - June 2022): 73.84%
```

This calculation was then replicated for North East and North Cumbria ICB, so the NHS could see their daily utilisation.

Question 4: What was NHS North East and North Cumbria ICB's average daily appointment capacity utilisation (%) for the period August 2021 to June 2022?

[62]:
```
# Calculate the average number of appointments booked per day between August 2021 to June 2022 in NHS North East and North Cumbria ICB.
# Use the nc_num_appt_days DataFrame for the total number of appointment dates.
avg_daily_nenc_dem = ar_no_dups_aug21_to_jun22_nenc['count_of_appointments'].sum() / nc_num_appt_days

# Create a new DataFrame to hold the average daily capacity utilisation calculation of NHS North East and North Cumbria ICB.
avg_daily_nenc_cap_utilisation_aug21_to_jun22 = avg_daily_nenc_dem / daily_icb_cap

# View the result.
print(f"NHS North East and North Cumbria ICB's average daily national appointment capacity utilisation (August 2021 - June 2022): \
{avg_daily_nenc_cap_utilisation_aug21_to_jun22 * 100:.2f}%")
```

```
NHS North East and North Cumbria ICB's average daily national appointment capacity utilisation (August 2021 - June 2022): 176.79%
```

DNA rates were then calculated by dividing the number of DNAs by the count of appointments, to determine the amount of wasted capacity.

Question 5: What proportions of the total national appointment demand between August 2021 and June 2022 were attended, booked, or status unknown?

[63]:
```
# Group the ar_no_dups_aug21_to_jun22 DataFrame by appointment_status and sum the count of appointments.
ar_no_dups_aug21_to_jun22_statuses = ar_no_dups_aug21_to_jun22.groupby('appointment_status')['count_of_appointments'].sum().reset_index()

# Add a percentage column to the ar_no_dups_aug21_to_jun22_statuses DataFrame.
ar_no_dups_aug21_to_jun22_statuses['percentage'] = (ar_no_dups_aug21_to_jun22_statuses['count_of_appointments'] / \
                                                    ar_no_dups_total_appts_aug21_to_jun22 * 100).round(2)

ar_no_dups_aug21_to_jun22_statuses
```

[63]:

	appointment_status	count_of_appointments	percentage
0	Attended	270617423	91.44
1	DNA	13285796	4.49
2	Unknown	12037388	4.07

Question 6: What proportions of NHS North East and North Cumbria ICB's total appointment demand between August 2021 and June 2022 were attended, booked, or status unknown?

[64]:
```
# Group the ar_no_dups_aug21_to_jun22_nenc DataFrame by appointment_status and sum the count of appointments.
ar_no_dups_aug21_to_jun22_nenc_statuses = ar_no_dups_aug21_to_jun22_nenc.groupby('appointment_status')['count_of_appointments'].sum().reset_index()

# Add a percentage column to the ar_no_dups_aug21_to_jun22_nenc_statuses DataFrame.
ar_no_dups_aug21_to_jun22_nenc_statuses['percentage'] = (ar_no_dups_aug21_to_jun22_nenc_statuses['count_of_appointments'] / \
                                                         ar_no_dups_total_appts_aug21_to_jun22_nenc * 100).round(2)

# View the output.
ar_no_dups_aug21_to_jun22_nenc_statuses
```

[64]:

	appointment_status	count_of_appointments	percentage
0	Attended	15498465	91.87
1	DNA	724157	4.29
2	Unknown	648148	3.84

DNA rates were then calculated for each appointment mode. This required the creation of new DataFrame to hold the DNA rates and the merging of this with the DataFrame that held the counts of appointments. This was necessary to determine whether any modes could be overbooked to offset wastage.

Question 7: What was NHS North East and North Cumbria ICB's DNA rate per appointment mode between August 2021 and June 2022?

[65]:
```
# Calculate the total number of appointments per appointment mode.
ar_no_dups_aug21_to_jun22_appt_mode_totals = ar_no_dups_aug21_to_jun22_nenc \
    .groupby('appointment_mode')['count_of_appointments'].sum().reset_index()

# Add the appointment mode and appt mode totals columns to the ar_no_dups_aug21_to_jun22_appt_mode_totals DataFrame.
ar_no_dups_aug21_to_jun22_appt_mode_totals.columns = ['appointment_mode', 'appt_mode_totals']

# Sum the DNA counts by appointment mode.
ar_no_dups_aug21_to_jun22_nenc_dna_counts_by_mode = ar_no_dups_aug21_to_jun22_nenc[ar_no_dups_aug21_to_jun22_nenc['appointment_status'] == 'DNA'] \
    .groupby('appointment_mode')['count_of_appointments'].sum().reset_index()

# Add the appointment mode and dna_count columns to the ar_no_dups_aug21_to_jun22_nenc_dna_counts_by_mode DataFrame.
ar_no_dups_aug21_to_jun22_nenc_dna_counts_by_mode.columns = ['appointment_mode', 'dna_count']

# Merge the ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_mode DataFrame with the DNA counts by appointment mode.
ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_mode = ar_no_dups_aug21_to_jun22_nenc_dna_counts_by_mode \
    .merge(ar_no_dups_aug21_to_jun22_appt_mode_totals, on='appointment_mode')

# Calculate the DNA rate per appointment mode.
ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_mode['dna_rate'] = (ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_mode['dna_count'] / \
                                                                ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_mode['appt_mode_totals'] * 100).round(2)

# Reorder the columns within the ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_mode DataFrame.
ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_mode = ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_mode \
    [['appointment_mode', 'appt_mode_totals', 'dna_count', 'dna_rate']]

# View results.
ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_mode
```

[65]:

	appointment_mode	appt_mode_totals	dna_count	dna_rate
0	Face-to-Face	11327961	609528	5.38
1	Home Visit	123916	4061	3.28
2	Telephone	4894010	92769	1.90
3	Unknown	481089	15623	3.25
4	Video/Online	43794	2176	4.97

Question 8: What was NHS North East and North Cumbria ICB's DNA rates for face-to-face and video/online appointments, month on month, between August 2021 and June 2022?

[66]:
```
# Filter the ar_no_dups_aug21_to_jun22_nenc DataFrame for face-to-face and video/online appointment modes only.
ar_no_dups_aug21_to_jun22_nenc_f2f_and_vo_appts = ar_no_dups_aug21_to_jun22_nenc[ar_no_dups_aug21_to_jun22_nenc['appointment_mode'] \
    .isin(['Face-to-Face', 'Home Visit', 'Telephone', 'Video/Online'])]

# Calculate the total number of appointments per appointment mode and appointment month.
ar_no_dups_aug21_to_jun22_nenc_appt_mode_monthly_totals = ar_no_dups_aug21_to_jun22_nenc_f2f_and_vo_appts \
    .groupby(['appointment_mode', 'appointment_month'])['count_of_appointments'].sum().reset_index()

# Add the appointment mode, appointment month, and appt mode month totals columns to the
# ar_no_dups_aug21_to_jun22_nenc_appt_mode_monthly_totals DataFrame.
ar_no_dups_aug21_to_jun22_nenc_appt_mode_monthly_totals.columns = ['appointment_mode', 'appointment_month', 'appt_mode_month_totals']

# Filter the ar_no_dups_aug21_to_jun22_nenc_f2f_and_vo_appts DataFrame for DNAs only.
ar_no_dups_aug21_to_jun22_nenc_dnas = ar_no_dups_aug21_to_jun22_nenc_f2f_and_vo_appts[ar_no_dups_aug21_to_jun22_nenc_f2f_and_vo_appts \
    ['appointment_status'] == 'DNA']

# Sum the DNA counts by appointment mode and appointment month.
ar_no_dups_aug21_to_jun22_nenc_dna_counts_by_month_and_mode = ar_no_dups_aug21_to_jun22_nenc_dnas \
    .groupby(['appointment_mode', 'appointment_month'])['count_of_appointments'].sum().reset_index()

# Add the appointment mode, appointment month, and DNA count columns to the ar_no_dups_aug21_to_jun22_nenc_dna_counts_by_month_and_mode DataFrame.
ar_no_dups_aug21_to_jun22_nenc_dna_counts_by_month_and_mode.columns = ['appointment_mode', 'appointment_month', 'dna_count']

# Merge the DNA counts with the total appointment counts.
ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_month_and_mode = ar_no_dups_aug21_to_jun22_nenc_dna_counts_by_month_and_mode \
    .merge(ar_no_dups_aug21_to_jun22_nenc_appt_mode_monthly_totals, on=['appointment_mode', 'appointment_month'])

# Calculate the DNA rate per appointment mode, per appointment month.
ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_month_and_mode['dna_rate'] = (ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_month_and_mode['dna_count'] / \
                                        ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_month_and_mode['appt_mode_month_totals'] * 100).round(2)

# Pivot the aggregated data.
pivot_ar_no_dups_aug21_to_jun22_nenc_dnas = ar_no_dups_aug21_to_jun22_nenc_dna_rates_by_month_and_mode \
    .pivot(index='appointment_month', columns='appointment_mode', values='dna_rate')

# Rename the appointment mode column header to DNA rate (%).
pivot_ar_no_dups_aug21_to_jun22_nenc_dnas.columns.name = 'dna_rate (%)'

# View the results.
pivot_ar_no_dups_aug21_to_jun22_nenc_dnas
```

[66]:

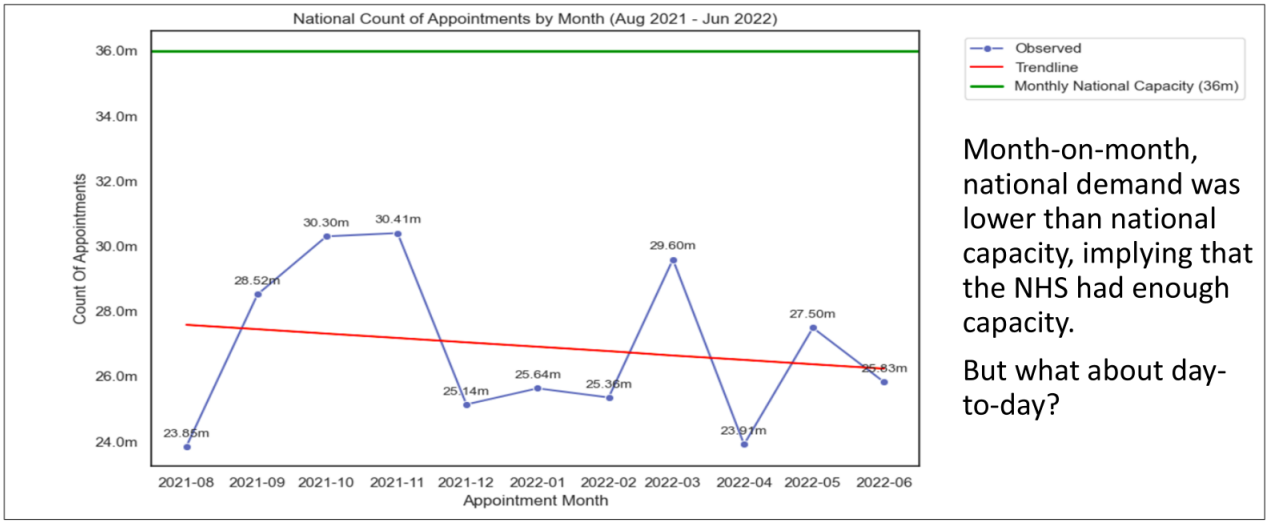
dna_rate (%)	Face-to-Face	Home Visit	Telephone	Video/Online
appointment_month				
2021-08	4.81	3.08	1.85	4.98
2021-09	5.24	2.16	1.78	4.02
2021-10	5.97	1.91	1.82	3.78
2021-11	5.40	2.51	1.80	4.90
2021-12	5.93	3.61	1.87	6.22
2022-01	5.29	3.55	1.77	4.93
2022-02	5.14	3.79	1.85	5.45
2022-03	5.31	4.15	1.96	5.82
2022-04	5.32	3.46	2.04	4.96
2022-05	5.25	4.17	2.04	5.28
2022-06	5.37	3.32	2.14	5.55

Data Visualisations

Exploratory visualisations were created to identify patterns and trends. These were then refined into explanatory visualisations.

To answer business question 1, the following visualisations were created:

Did the NHS have enough monthly capacity at a national level?

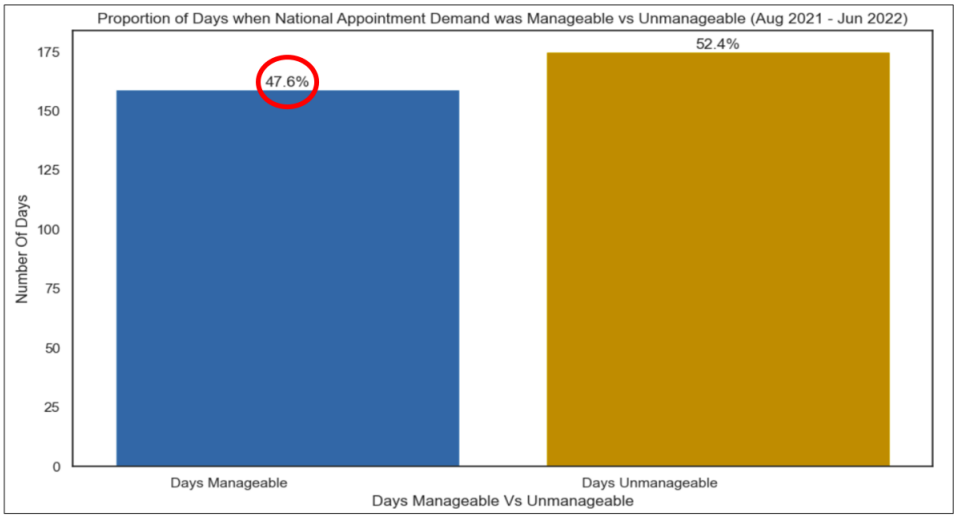


Month-on-month, national demand was lower than national capacity, implying that the NHS had enough capacity.

But what about day-to-day?

This timeseries chart was chosen because it clearly shows monthly demand vs. capacity. It informs us that demand was below capacity during each month. This implies that the NHS had enough capacity to manage demand.

Did the NHS have enough daily capacity at a national level?



Nationally, the NHS only had enough capacity to manage its demand on 47.6% of the observed days – fewer than half!

But how did individual ICBs fare each day?

This bar chart was chosen for easy comparison of two outcomes. It informs us that demand was manageable on 47.6% of the observed days. This suggests that the NHS had sufficient capacity to manage its demand less than half the time.

Which ICBs consistently had insufficient capacity?

ICBs with Insufficient Capacity	NHS Region
NHS North East London	London
NHS North West London	
NHS North East and North Cumbria	North East and Yorkshire
NHS West Yorkshire	
NHS Cheshire and Merseyside	North West
NHS Greater Manchester	

Six of the 42 ICBs consistently had insufficient capacity to manage their daily demand.

But could any of the other 36 ICBs have helped them?

Which ICBs had unutilised capacity in the London, North East and Yorkshire, and North West regions, and how much did they have?

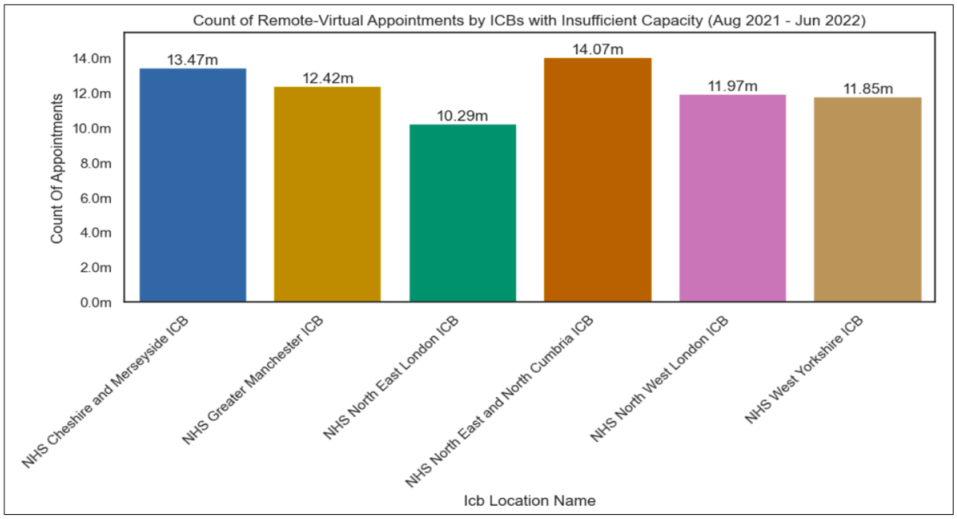
ICBs with Insufficient Capacity	NHS Region	ICBs with Sufficient Capacity	Unutilised Capacity
NHS North East London	London	NHS North Central London	2,680,472
NHS North West London		NHS South East London	1,679,260
		NHS South West London	2,274,800
NHS North East and North Cumbria	North East and Yorkshire	NHS Humber and North Yorkshire	519,653
NHS West Yorkshire		NHS South Yorkshire	2,079,015
NHS Cheshire and Merseyside	North West	NHS Lancashire and South Cumbria	787,008
NHS Greater Manchester			
Total unutilised capacity over the total 11 months observed			**10,020,208**

Six other ICBs belonging to the same regions as those with insufficient capacity had more than 10 million unutilised appointments between them.

Could any of the demand been shifted to them?

These tables were used because they neatly present the ICBs within scope of the analytical question. The first table informs the NHS which ICBs consistently had insufficient capacity and the second shows which neighbouring ICBs had unutilised capacity.

How much demand was there for remote/virtual appointments?



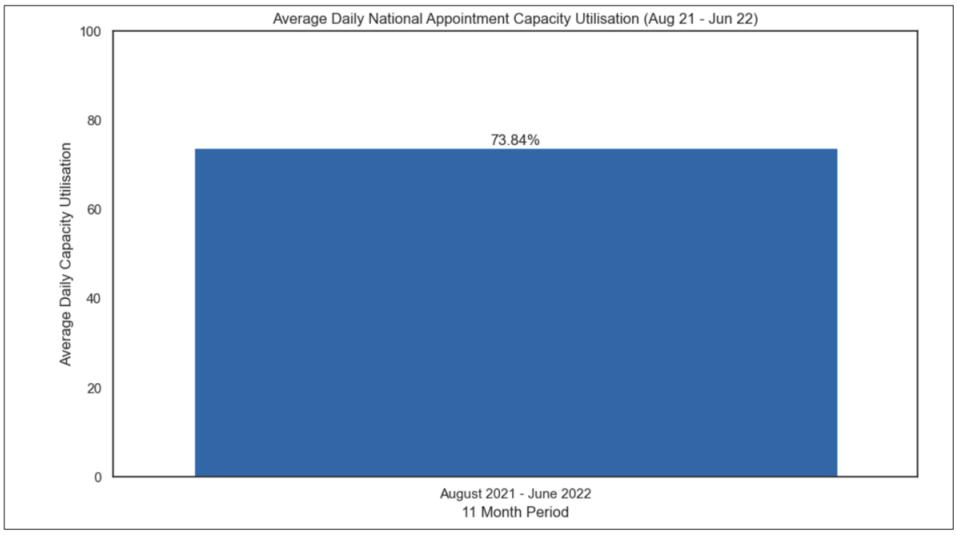
Each ICB with insufficient capacity experienced demand for between 10 and 14 million remote/virtual appointments.

Some of these potentially could have been shifted to the neighbouring ICBs.

This bar chart was chosen to enable easy comparison of ICBs. It informs us that these six ICBs each experienced demand for 10 to 14 million remote/virtual appointments.

To answer business question 2, the following visualisations were created:

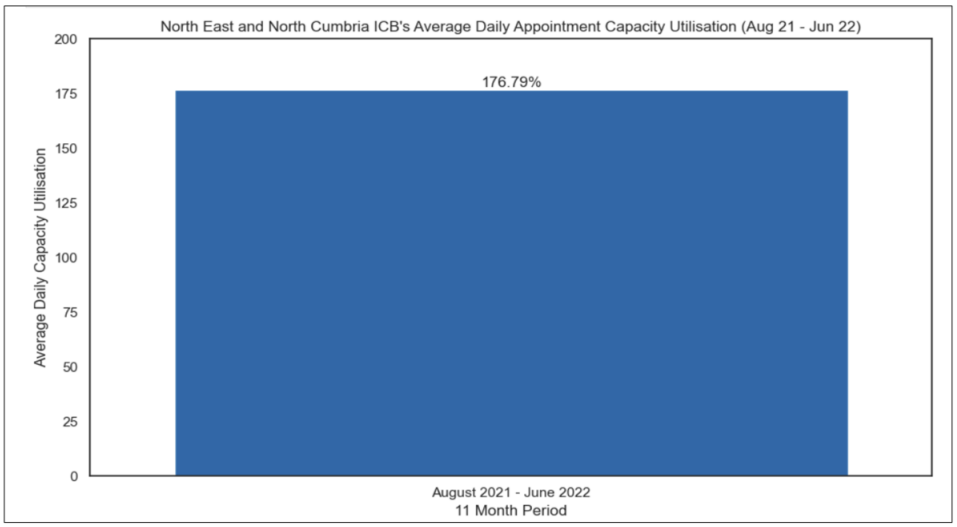
What was the average daily national capacity utilisation?



On an average day, the NHS utilised only 74% of its total national capacity.

But how much capacity did NHS North East and North Cumbria ICB utilise?

What was NHS North East and North Cumbria ICB's average daily capacity utilisation?

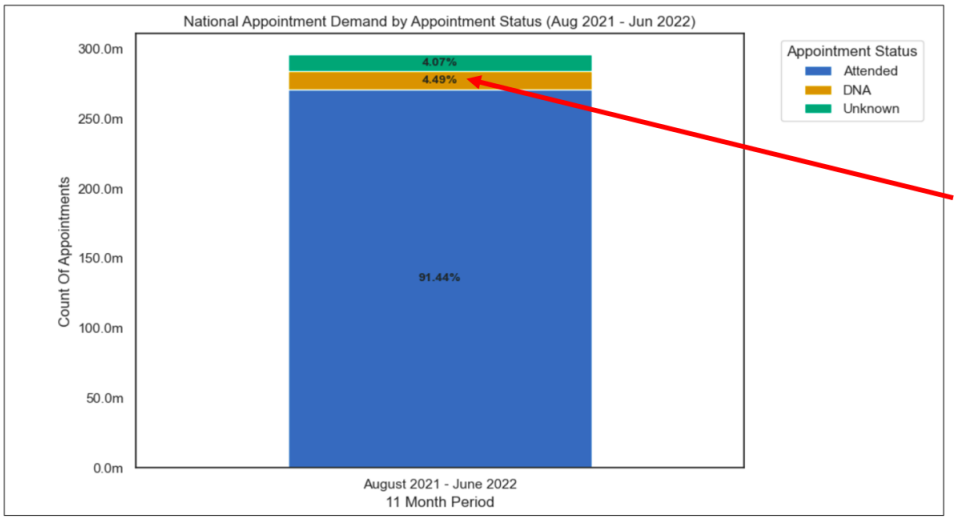


On an average day, demand outstripped capacity by 77% in NHS North East and North Cumbria.

But what proportion of booked appointments were actually attended by patients?

These bar charts were chosen because the height of the columns can easily be compared with the percentages on the y-axes. They show that the NHS used 74% of its total national capacity, whilst North East and North Cumbria ICB's demand outstripped capacity by 77%.

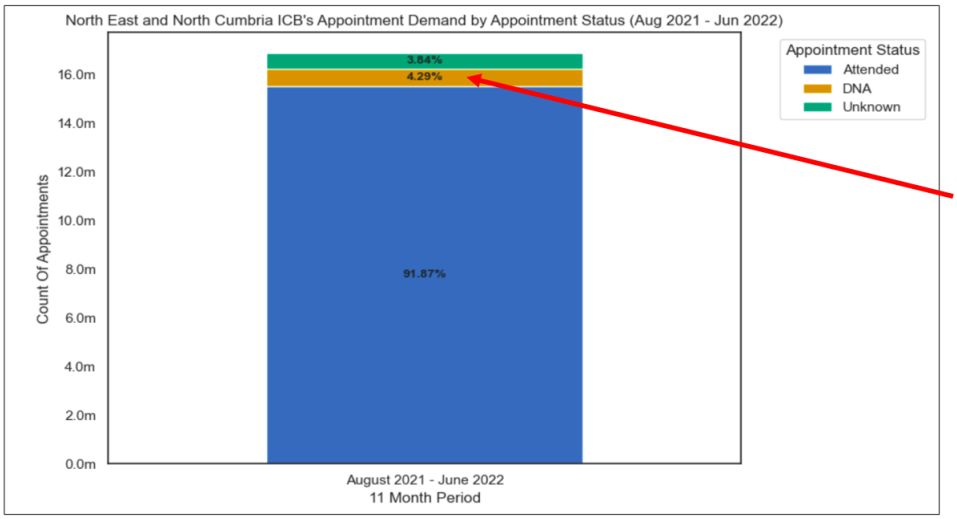
What proportion of total national appointments were attended, DNAs, or had an unknown appointment status?



4.5% of the total appointments booked nationally were DNAs – 1/22 appointments were wasted!

How did NHS North East and North Cumbria ICB compare?

What proportion of NHS North East and North Cumbria's appointments were attended, DNAs, or had an unknown appointment status?

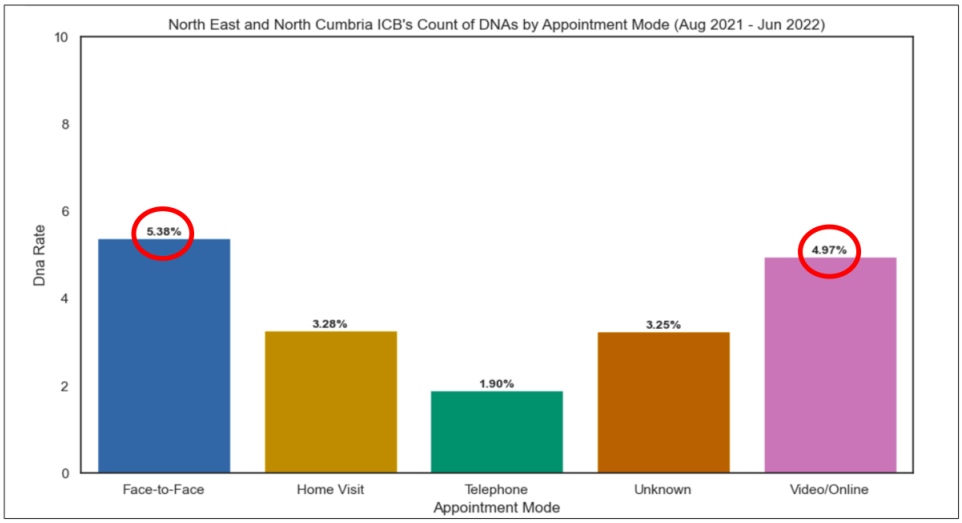


4.3% of North East and North Cumbria's booked appointments were DNAs – 1/23 appointments were wasted!

Which appointment modes had the highest DNA rates?

These stacked bar charts were chosen to enable easy comparison between three different outcomes. They show that the overall NHS and North East and North Cumbria ICB's experienced comparable DNA rates.

What were the DNA rates of each appointment mode in NHS North East and North Cumbria ICB?

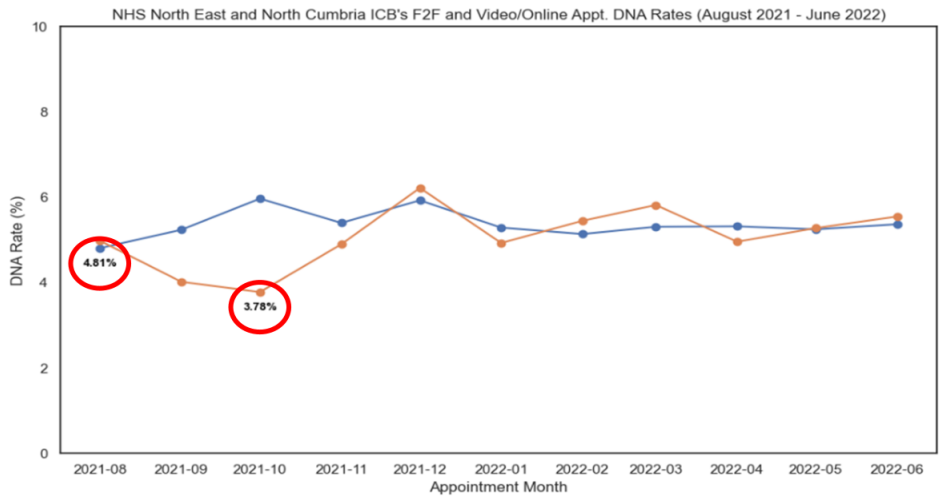


The DNA rate was approximately 5% for both total face-to-face and total video/online appointments.

But what were the month-on-month DNA rates for these appointment modes?

This bar chart was chosen to enable easy comparison between appointment modes. It informs us that approximately 5% of NHS North East and North Cumbria's face-to-face and video/online appointments were DNAs.

What were the monthly DNA rates of face-to-face and video/online appointments in NHS North East and North Cumbria?

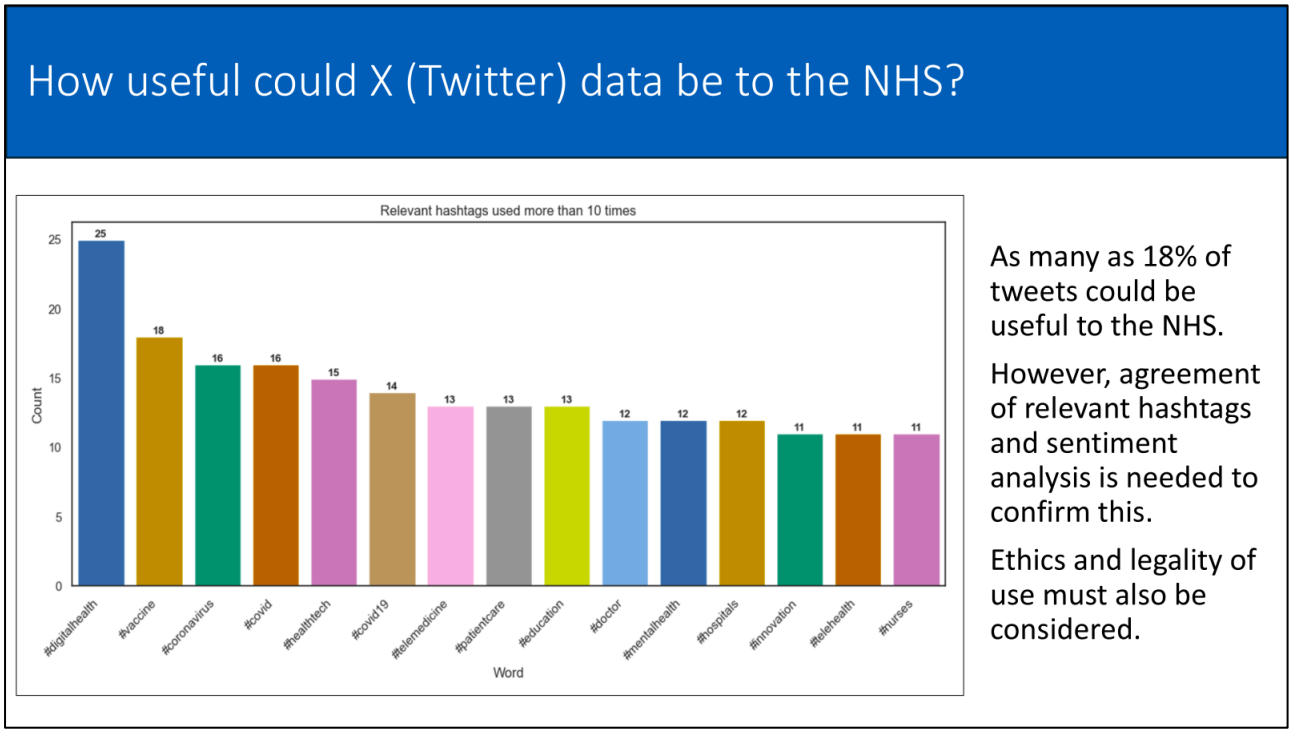


The lowest DNA rates for face-to-face and video/online appointments were 4.81% and 3.78%.

Some of the wasted appointments could potentially have been recovered through overbooking strategies.

This timeseries chart was chosen because it clearly shows month-on-month DNAs. It informs us that the lowest DNA rate for face-to-face appointments was 4.81%, and the lowest for video/online appointments was 3.78%.

X (Twitter) data analysis:



This bar chart was chosen to enable easy comparison of hashtags. It informs us that 16 relevant hashtags were used more than 10 times each and that these featured in 18% of the sample tweets.

<u>Recommendations and Further Actions</u>

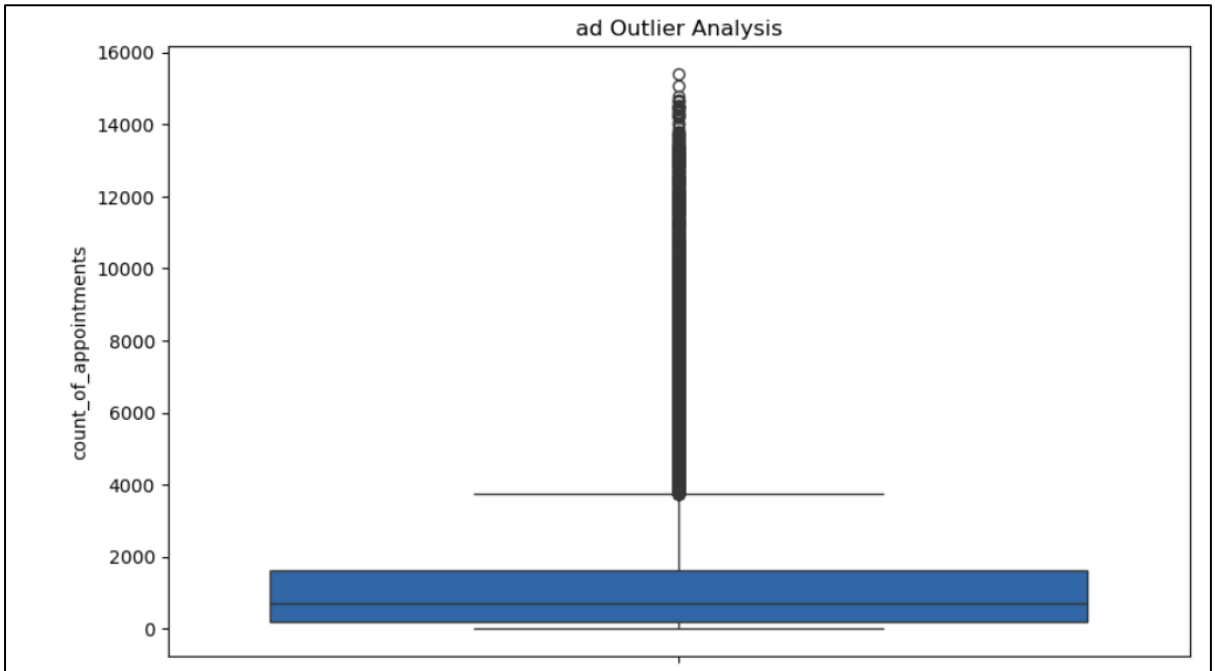
It is recommended that the NHS repeats this analysis after the financial year 2026/27. This is because geographic boundaries changed on 1st April 2026, meaning a full year will need to pass before primary care data that reflects these changes is available.

The analysis could be strengthened by incorporating either funding or population size data to more accurately assess each ICB's available capacity.

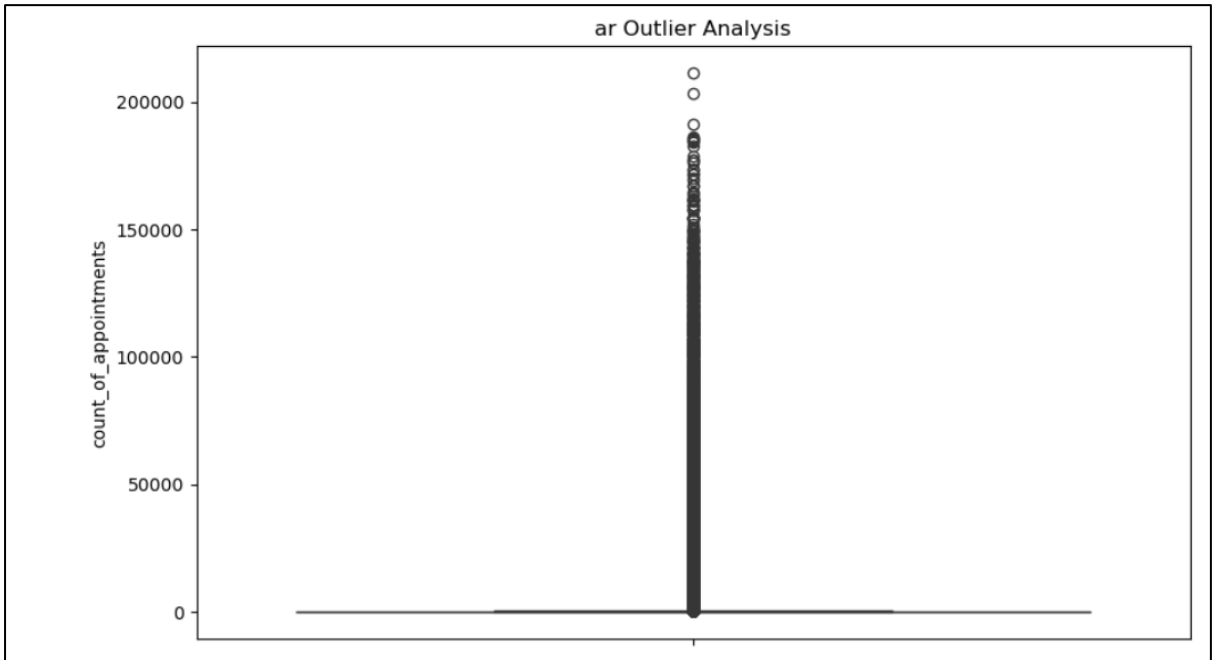
Data from the National Workforce Reporting System and the Additional Roles Reimbursement Scheme could also be incorporated, as it includes the whole-time equivalents of different staff employed in general practice and primary care networks, and could, therefore, help determine whether there are adequate staff in primary care.

Appendices

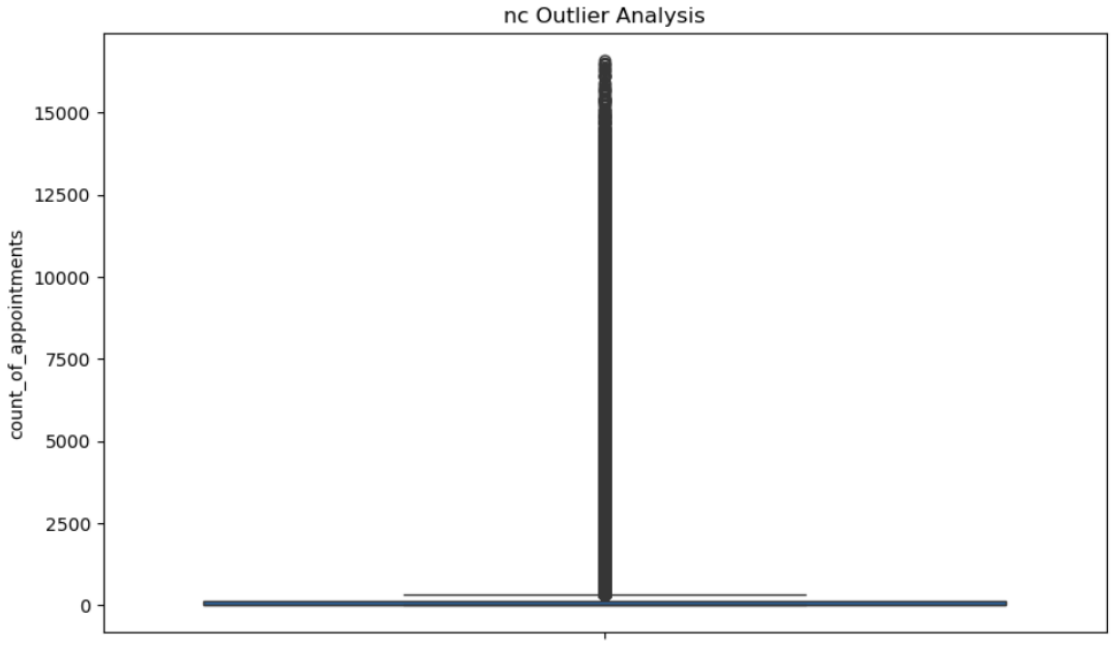
Appendix 1: Outlier analyses and boxplots of the ad, ar, and nc datasets:




```
[9]: {'Q1': np.float64(194.0),
      'Q3': np.float64(1621.0),
      'IQR': np.float64(1427.0),
      'lower_bound': np.float64(-1946.5),
      'upper_bound': np.float64(3761.5),
      'outlier_count': 9291}
```




```
[11]: {'Q1': np.float64(7.0),
       'Q3': np.float64(308.0),
       'IQR': np.float64(301.0),
       'lower_bound': np.float64(-444.5),
       'upper_bound': np.float64(759.5),
       'outlier_count': 97348}
```




```
[13]: {'Q1': np.float64(7.0),
       'Q3': np.float64(128.0),
       'IQR': np.float64(121.0),
       'lower_bound': np.float64(-174.5),
       'upper_bound': np.float64(309.5),
       'outlier_count': 147958}
```

Appendix 2: Analytical questions, outputs, and visualisations that helped answer the business questions, but which were not included in the stakeholder presentation.

Business Question 1

Question 1: How many appointments were available each month nationally (capacity)?

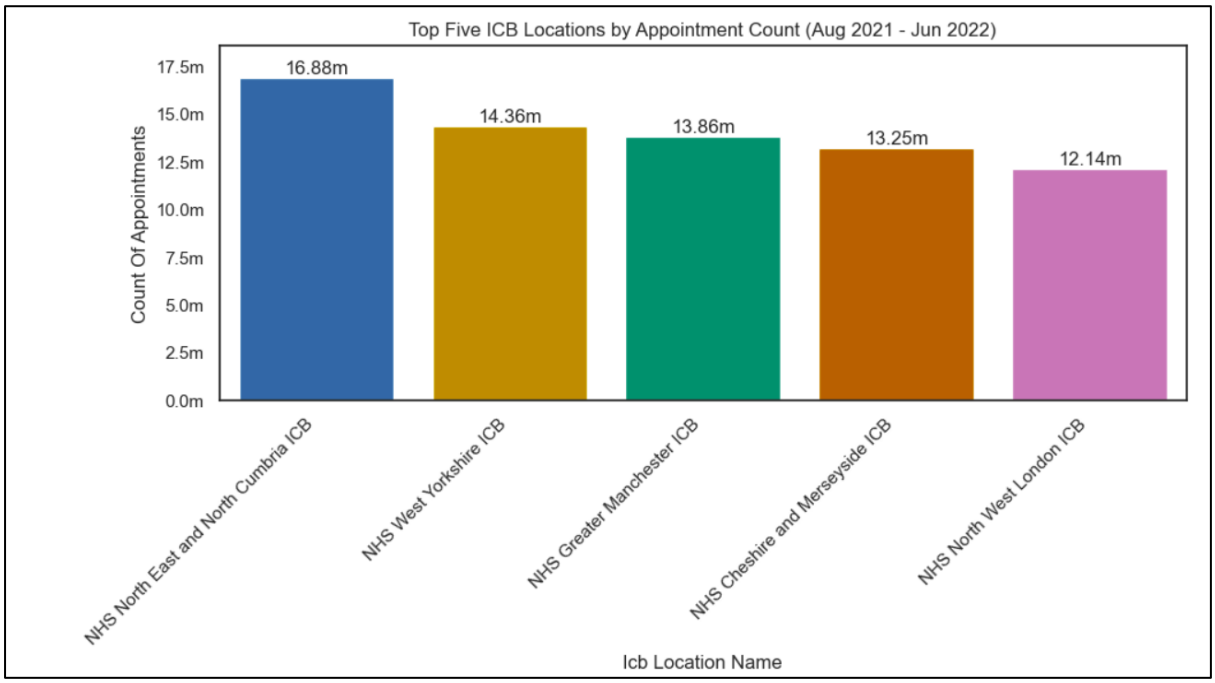
[45]:
```
# The assignment details inform us that the daily national appointment capacity is 1.2 million.

# Create a DataFrame to store the daily national appointment capacity.
daily_nat_cap = 1200000

# Create a DataFrame to calculate and store the monthly national capacity.
monthly_nat_cap = daily_nat_cap * 30

# View the result.
print(monthly_nat_cap)
```

```
36000000
```

Question 4: What were the top five ICB locations by appointment count?

[49]:
```
# Determine the top five locations based on appointment count.
nc.groupby("icb_location_name")['count_of_appointments'].sum().nlargest(5)
```

[49]:
```
icb_location_name
NHS North East and North Cumbria ICB    16882235
NHS West Yorkshire ICB                  14358371
NHS Greater Manchester ICB              13857900
NHS Cheshire and Merseyside ICB         13250311
NHS North West London ICB               12142390
Name: count_of_appointments, dtype: int64
```



Question 5: What was each ICB's monthly capacity?

[50]:
```
# We are not told the proportion of the daily national capacity that each ICB location has locally.
# Therefore, it is assumed that each ICB location has 1/42 of the daily national capacity (an equal share).

# Create a DataFrame to store the daily ICB location appointment capacity.
daily_icb_cap = round(1200000 / 42)

# Create a DataFrame to calculate and store the monthly ICB location appointment capacity.
monthly_icb_cap = round(daily_icb_cap * 30)

# View the result as a whole number.
print(monthly_icb_cap)
```

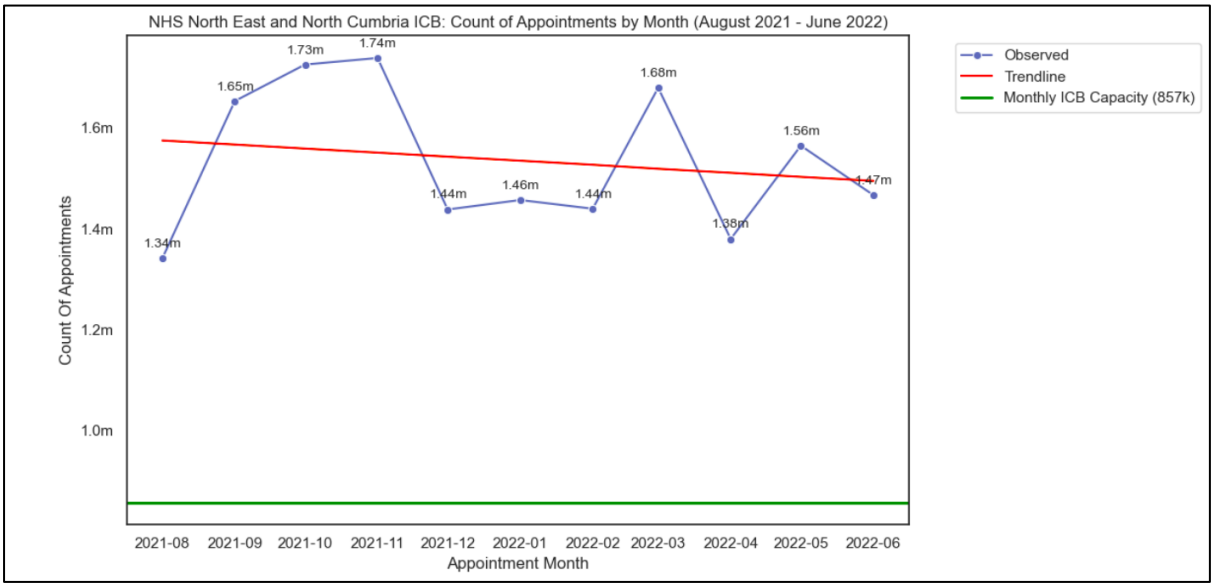
```
857130
```

Question 6: How many appointments were booked in the top ICB location by appointment count each month (demand)?

[51]:
```
# The top ICB location by appointment amount was NHS North East and North Cumbria ICB.
# Create a subset of the nc dataset to determine the number of appointments booked per month in NHS North East and North Cumbria ICB.
nenc_icb_subset = nc[(nc['icb_location_name'] == 'NHS North East and North Cumbria ICB')]

# Group the data by month and sum the count of appointments.
nenc_icb_monthly_appt_counts = nenc_icb_subset.groupby('appointment_month')['count_of_appointments'].sum().reset_index()

# View the results.
nenc_icb_monthly_appt_counts
```

[51]:

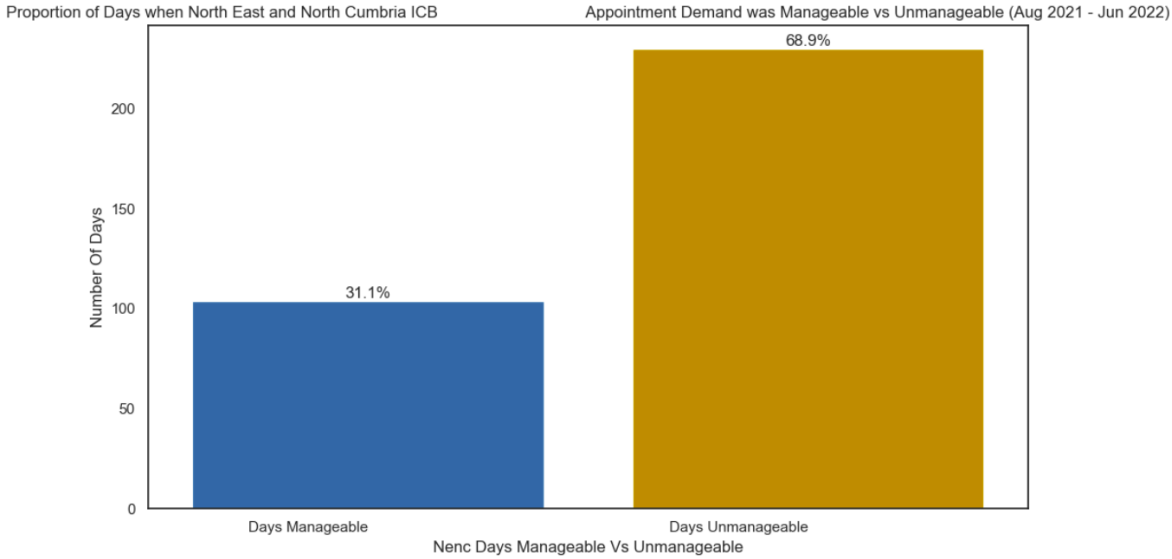
	appointment_month	count_of_appointments
0	2021-08	1341051
1	2021-09	1653075
2	2021-10	1725403
3	2021-11	1738485
4	2021-12	1437989
5	2022-01	1456780
6	2022-02	1439488
7	2022-03	1679342
8	2022-04	1379494
9	2022-05	1564541
10	2022-06	1466587



Question 7: How often was the top ICB location by appointment count's demand manageable within its capacity?

[52]:
```
# The top ICB location by appointment amount was NHS North East and North Cumbria ICB.
# Calculate the number of days on which appointments were booked.
nenc_num_appt_days = nenc_icb_subset['appointment_date'].nunique()

# View the result.
print(f"Number of days on which appointments were booked: {nenc_num_appt_days}")

# Group by appointment date and sum all appointments in NHS North East and North Cumbria ICB.
daily_nenc_dem = nenc_icb_subset.groupby('appointment_date')['count_of_appointments'].sum()

# Filter the appointment dates to determine the number of days where demand was manageable within NHS North East and North Cumbria ICB's capacity.
days_sufficient_nenc_cap = daily_nenc_dem[daily_nenc_dem < daily_icb_cap]

# View the result.
print(f"Number of days with sufficient capacity in NHS North East and North Cumbria ICB to meet demand: {len(days_sufficient_nenc_cap)}")

# Calculate the percentage of total days where demand was manageable within NHS North East and North Cumbria ICB's capacity.
perc_days_sufficient_nenc_cap = round((len(days_sufficient_nenc_cap) / nenc_num_appt_days) * 100, 2)

# View the result.
print(f"Percentage of days with sufficient capacity in NHS North East and North Cumbria ICB to meet demand: {perc_days_sufficient_nenc_cap}")
```

```
Number of days on which appointments were booked: 334
Number of days with sufficient capacity in NHS North East and North Cumbria ICB to meet demand: 104
Percentage of days with sufficient capacity in NHS North East and North Cumbria ICB to meet demand: 31.14
```



Business Question 2

Question 1: What was the total national appointment capacity utilisation (%) for the period August 2021 to June 2022?

[59]:
```
# Filter the ar_no_dups DataFrame for the appointment months August 2021 to June 2022.
ar_no_dups_aug21_to_jun22 = ar_no_dups[(ar_no_dups['appointment_month'] >= '2021-08') & (ar_no_dups['appointment_month'] <= '2022-06')]

# Sum the count_of_appointments column.
ar_no_dups_total_appts_aug21_to_jun22 = ar_no_dups_aug21_to_jun22['count_of_appointments'].sum()

# Create a new DataFrame to hold the national capacity utilisation calculation.
# There are 11 months between August 2021 and June 2022, so monthly_nat_cap will be multiplied by 11 to get the total capacity for this period.
nat_cap_utilisation_aug21_to_jun22 = ar_no_dups_total_appts_aug21_to_jun22 / (monthly_nat_cap * 11)

print(f"National appointment capacity utilisation (August 2021 - June 2022): {nat_cap_utilisation_aug21_to_jun22 * 100:.2f}%")
```

```
National appointment capacity utilisation (August 2021 - June 2022): 74.73%
```

Question 3: What was NHS North East and North Cumbria ICB's total appointment capacity utilisation (%) for the period August 2021 to June 2022?

[61]:
```
# Filter the ar_no_dups DataFrame for the appointment months August 2021 to June 2022 for NHS North East and North Cumbria ICB.
ar_no_dups_aug21_to_jun22_nenc = ar_no_dups[(ar_no_dups['appointment_month'] >= '2021-08') & (ar_no_dups['appointment_month'] <= '2022-06') \
    & (ar_no_dups['icb_location_name'] == 'NHS North East and North Cumbria ICB')]

# Sum the count_of_appointments column.
ar_no_dups_total_appts_aug21_to_jun22_nenc = ar_no_dups_aug21_to_jun22_nenc['count_of_appointments'].sum()

# Create a new DataFrame to hold NHS North East and North Cumbria ICB's capacity utilisation calculation.
# There are 11 months between August 2021 and June 2022, so monthly_icb_cap will be multiplied by 11 to get the total capacity for this period.
nenc_cap_utilisation_aug21_to_jun22 = ar_no_dups_total_appts_aug21_to_jun22_nenc / (monthly_icb_cap * 11)

print(f"NHS North East and North Cumbria ICB's appointment capacity utilisation (August 2021 - June 2022): \
{nenc_cap_utilisation_aug21_to_jun22 * 100:.2f}%")
```

```
NHS North East and North Cumbria ICB's appointment capacity utilisation (August 2021 - June 2022): 178.94%
```